A04b · The Notation That Survives — Practice A2 · 9618 §19.1e

Name: _____ Due: _____

Big O is a **Paper 3** topic — it has never once appeared on a Paper 4. So the written half is the exam half, and the code half is **yours**: no mark points, no CIE conventions, just an instrument that tells the truth. **Questions 1–6 are on this sheet; questions 7–9 are in `A04b_Practice.py`.** One set of numbers across both.

1. Read the order off the code. Give the Big O of each. Write them like `O(n)`; spacing does not matter. [5]

(a)

```
for x in A:
    total = total + x
```

(b)

```
for x in A:
    for y in A:
        pairs = pairs + 1
```

(c)

```
i = len(A)
while i > 1:
    i = i // 2
```

(d)

```
for k in range(100):
    total = total + A[0]
```

(e) — this one is the interesting one.

```
for x in A:
    i = len(A)
    while i > 1:
        i = i // 2
```

2. The case is half the claim. [4]

(a) Complete the table. [3]

algorithm	best case	worst case
insertion sort *(done for you)*	$O(n)$	$O(n^2)$
linear search		
binary search	$O(1)$	

(b) A **selection sort** does the same number of comparisons on already-sorted data as on random data — its best case and worst case are the same. Explain why. [1]

3. Both searches, as the data grows. Two examples of searching algorithms are a linear search and a binary search. [3]
Describe how **increasing the number of data items** in the file to be searched affects the **time efficiency** of these search algorithms. **Refer to Big O notation in your answer.**

4. Read the marks before you write. Here is a real exam question, in full: [3]

> "Explain how the performance of a binary search varies according to the number of values in the array." **[1]**

(a) Write an answer worth exactly one mark. [1]

(b) A candidate writes half a page about halving the array and discarding one side, and scores the same one mark. State **what the extra writing would have earned marks for**, if the question had been worth three. [2]

5. Which one would you ship? Two programs do the same job. [5]

> Algorithm **A** costs $2n^2$ operations · Algorithm **B** costs $50n$ operations

(a) Complete the table. [2]

n	A = $2n^2$	B = $50n$	which does less work?
10			
1 000			

(b) Find the value of n at which the two cost exactly the same. Show your working. [2]

(c) **B** has the better Big O. Give **one** reason a programmer might still choose **A**. [1]

6. The other axis. Big O measures memory as well as time. [2]

(a) Complete this sentence, which is worth a mark on its own in more than one published mark scheme:

"Big O notation is used to indicate the

of an algorithm." [1]

(b) A hash table finds an item in `O(1)` — faster than any search on this sheet. State what it **pays** for that speed. [1]

On the computer — `A04b_Practice.py`

Fill in the three methods and run it — the judge is at the bottom of the file. **There is no mark scheme behind these**, because Big O is never examined at a keyboard. They are here because a claim you have measured yourself is a claim you own.

7	`linearComparisons` — search, and count what it cost	[3]
8	`binaryComparisons` — the same, halving	[3]
9	`growthClass` — name the order from two measurements alone	[2]

Written 22 + code 8 = **30 marks**. The judge finishes by running *your* questions 7 and 8 at n and 2n, and printing the ratio.

Want to dig? Question 1(e) is a halving loop *inside* a pass — `n` times $\log_2 n$. Double the array and the operation count goes up by a factor of about **2.2**, not 2 and not 4. Check that: count the operations at `n = 1000` and at `n = 2000` and divide. Then answer the harder question — **that ratio is not a constant.** Work out what it becomes at a million, and say what it is creeping towards, and why that means `n log n` behaves almost like `O(n)` on any data you will ever own.

Additional space. If you need more room for any answer, continue here and write the question number in the margin beside it.